Done. I inspected both the proposal and the codebase, then added the resume/checkpoint infrastructure without changing the UC-VT/SAC/TAVT mathematical code. The proposal explicitly defines the ensemble uncertainty mechanism and soft weighting (w(u)=\exp(-\beta u/c_t)), while preserving the task-preserving/state-regularization components.

Updated package

Download the complete updated UC-VT codebase

I changed/added only the infrastructure files:

- `train.py`
- `config.py`
- `evaluate.py` — newly separated standalone evaluator
- `utils/checkpoint.py` — new checkpoint manager

I verified that `algorithm.py`, `networks.py`, `losses.py`, `envs.py`, and `buffers.py` are byte-for-byte unchanged from your uploaded version.

What is now supported

`python train.py ... --resume` automatically finds the newest standard or emergency checkpoint and restores the encoder, K-decoder ensemble, actor, twin critics, target critics, entropy parameter, **all optimizer states**, AMP `GradScaler`, UC-VT uncertainty history, Python/NumPy/PyTorch/CUDA RNG states, the dedicated `np.random.Generator`, iteration/episode counters, train/eval histories, training-task definitions and latents, and the replay buffer.

Full checkpoints are now written atomically and only the newest **3 standard checkpoints** remain. `best_model.pth` is separate and is never affected by rotation. The best score is defined as the mean discounted return across `near`, `mid`, and `far` OOD bins.

Loading always uses:

```
torch.load(path, map_location="cpu", weights_only=False)
```

before state dictionaries are copied to the GPU, avoiding the CUDA allocation spike that can occur when directly deserializing a GPU checkpoint.

`SIGTERM` and `SIGINT` are also trapped. They write:

```
emergency_checkpoint.pth
```

The emergency checkpoint intentionally excludes the large replay buffer so it has a much better chance of finishing before Kaggle terminates the process. When resuming from it, the newer model/RNG/counters come from the emergency checkpoint and the replay buffer is recovered from the most recent full checkpoint.

One unavoidable OS limitation: **SIGKILL cannot be trapped by any Python program**. SIGTERM/SIGINT are handled; a direct `kill -9` cannot trigger a save.

Kaggle commands

For a clean run, I recommend separate checkpoint directories for the two environments:

```
unzip -q ucvt_deep_rl_resume.zip -d /kaggle/working
cd /kaggle/working/ucvt_deep

pip install -r requirements.txt

python train.py \
  --task_family cheetah_vel \
  --total_iterations 2000 \
  --checkpoint_dir /kaggle/working/checkpoints/cheetah_vel \
  --log_dir /kaggle/working/logs/cheetah_vel
```

After a disconnect/restart, put the previous checkpoint folder back under `/kaggle/working/` if necessary, then simply run:

```
python train.py \
  --task_family cheetah_vel \
  --total_iterations 2000 \
  --resume \
  --checkpoint_dir /kaggle/working/checkpoints/cheetah_vel \
  --log_dir /kaggle/working/logs/cheetah_vel
```

`2000` still means the **target total iteration**, not 2000 additional iterations. For example, if the checkpoint finished iteration 836, it continues at 837 and stops at 2000.

You can also explicitly resume a particular checkpoint:

```
python train.py \
  --task_family cheetah_vel \
  --total_iterations 2000 \
```

```
  --resume_from
/kaggle/working/checkpoints/cheetah_vel/ckpt_it000836_ep0003348.pth
```

And evaluate the retained best model with:

```
python evaluate.py \
  --task_family cheetah_vel \
  --checkpoint best \
  --checkpoint_dir /kaggle/working/checkpoints/cheetah_vel
```

Or the most recent model:

```
python evaluate.py \
  --task_family cheetah_vel \
  --checkpoint latest \
  --checkpoint_dir /kaggle/working/checkpoints/cheetah_vel
```

Verification I performed

I ran syntax compilation on every modified Python file, a checkpoint-manager test covering save → rotate → mutate model → CPU-load → restore model/optimizer/replay state, a standard checkpoint resume test, an injected `SIGTERM` test that successfully generated `emergency_checkpoint.pth`, an emergency-checkpoint resume test using the previous full replay buffer, and a standalone `evaluate.py` checkpoint-load test.

The real MuJoCo smoke test could not be executed in this environment because `gymnasium` / `mujoco` are not installed and external package downloads are blocked here. I therefore exercised the entire train/resume/evaluate infrastructure using a temporary Gym-compatible test environment; that stub is **not** included in your package.

There are also two important **research-level gaps already present in the original implementation** that I intentionally did not alter. The proposal calls for an uncertainty-controlled curriculum that expands (\rho_t) according to calibration, but the implementation currently uses the fixed `virtual_interp_range=(-0.5, 1.5)`. Also, the proposal's final evaluation promises CVaR@20%, critic-target bias, uncertainty calibration, overhead, several baselines, and ablations; the current code evaluates primarily mean discounted return across the OOD bins. I left those untouched because your instruction was specifically to add infrastructure without altering the Deep RL mechanics.